For Mac

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Real Coder – Docker & Validation Instructions for Mac Users

Document Purpose

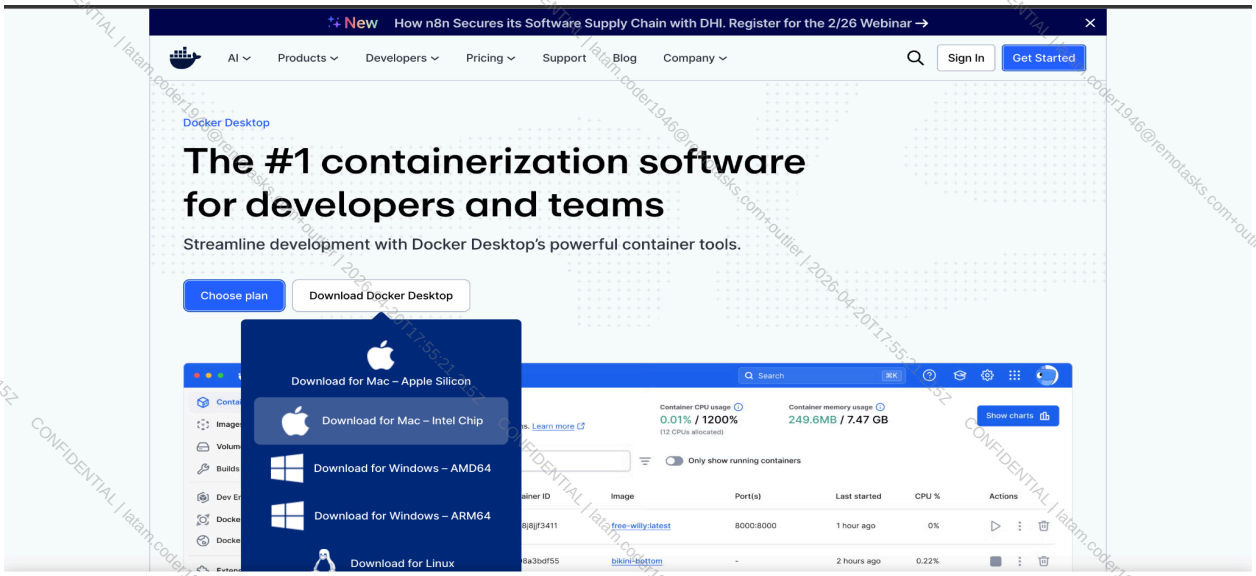
This guide explains how to safely modify the **Dockerfile**, work with **run.sh** and **parsing.py**, and run the validation process on a Mac computer. It covers the strict rules from the project instructions and step-by-step commands using **Docker Desktop**.

Important Warnings

- Follow **all** instructions **precisely**.
- The validation script depends on a **strict structure**.
- Any unintended changes (especially to Dockerfile structure, entrypoints, or default commands) **will break** the evaluation process.
- Only modify Dockerfile to **install required dependencies** for your solution and tests.
- **Never** use the COPY command in Dockerfile to add your code - dependencies only via apt, pip, etc.

Section 1: Prerequisites on Mac

1. Install **Docker Desktop** (required for building and running Docker on Mac).
    - Go to: https://www.docker.com/products/docker-desktop/
    - Download the version for your Mac (Apple Silicon M1/M2/M3/M4 or Intel).
    - Double-click the .dmg file → Drag Docker icon to Applications folder.
    - Launch Docker Desktop from Applications (it starts the Docker daemon).
    - Accept any permission prompts (it may ask for admin password).



CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

2. Open **Terminal** app on Mac (Applications → Utilities → Terminal).

Verify Docker works:

```
docker --version

docker info --format '{{.ServerVersion}}'
```

- If Docker is not running → open Docker Desktop first.

Section 2: Allowed & Forbidden Modifications to Dockerfile

Use **only** the provided Dockerfile template (from the project files).

Allowed changes

- Add new packages in the **SYSTEM DEPENDENCIES** section using apt-get install.
- Add Python packages using pip install (after python3 -m pip install --no-cache-dir).
- Example safe additions:

Dockerfile -

```
# After existing apt installs
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# After python-is-python3
RUN python3 -m pip install --no-cache-dir \
    requests \
    pytest \
    numpy \
    pandas
```

Forbidden changes – DO NOT DO THESE

- Modify the structure (sections, comments, order).
- Remove any existing lines/instructions.
- Add unrelated RUN, COPY, ADD, or other layers.
- Change FROM ubuntu:22.04, WORKDIR /app, ENV DEBIAN_FRONTEND=noninteractive.
- Change any entrypoint or default command (if present).

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

app > Dockerfile > ...

```dockerfile
##############################################
# BASE IMAGE
##############################################
FROM ubuntu:24.04 (last pushed 1 week ago)

ENV DEBIAN_FRONTEND=noninteractive

##############################################
# SYSTEM DEPENDENCIES (minimal to avoid disk-full on build)
# python3.14-venv only; no -dev to avoid pulling build-essential/gcc
##############################################
RUN apt-get update && apt-get install -y \
    git \
    curl \
    unzip \
    openssh-client \
    software-properties-common \
    && add-apt-repository ppa:deadsnakes/ppa \
    && apt-get update && apt-get install -y \
    python3.14 \
    python3.14-venv \
    python3-pip \
    && rm -rf /var/lib/apt/lists/*

##############################################
# CREATE VENV AT /opt/venv WITH ALL DEPENDENCIES
##############################################
RUN python3.14 -m venv /opt/venv \
    && /opt/venv/bin/pip install --upgrade pip "setuptools>=61.0" \
    && /opt/venv/bin/pip install --no-cache-dir \
        "click>=8.0" \
        "paramiko>=3.0" \
        "pyyaml>=6.0" \
        "rich>=13.0" \
        "pytest>=9.0.2"

ENV PATH="/opt/venv/bin:${PATH}"

# Point python3 and python to the venv interpreter so run.sh's
# fallback (python3) always resolves to the venv with all packages.
RUN ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python3 \
    && ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python

##############################################
# WORKING DIRECTORY + GIT SETUP
##############################################
WORKDIR /app

# PYTHONPATH includes /app so the log_aggregator package is importable
# once code is injected into /app at runtime (no COPY — code is volume-mounted
# or injected by the evaluation script).
ENV PYTHONPATH=/app

RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
    && git add README.md \
    && git commit -m "Initial commit"
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

app > Dockerfile > ...

```dockerfile
    && add-apt-repository ppa:deadsnakes/ppa \
    && apt-get update && apt-get install -y \
    python3.14 \
    python3.14-venv \
    python3-pip \
    && rm -rf /var/lib/apt/lists/*

##############################################
# CREATE VENV AT /opt/venv WITH ALL DEPENDENCIES
##############################################
RUN python3.14 -m venv /opt/venv \
    && /opt/venv/bin/pip install --upgrade pip "setuptools>=61.0" \
    && /opt/venv/bin/pip install --no-cache-dir \
        "click>=8.0" \
        "paramiko>=3.0" \
        "pyyaml>=6.0" \
        "rich>=13.0" \
        "pytest>=9.0.2"

ENV PATH="/opt/venv/bin:${PATH}"

# Point python3 and python to the venv interpreter so run.sh's
# fallback (python3) always resolves to the venv with all packages.
RUN ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python3 \
    && ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python

##############################################
# WORKING DIRECTORY + GIT SETUP
##############################################
WORKDIR /app

# PYTHONPATH includes /app so the log_aggregator package is importable
# once code is injected into /app at runtime (no COPY — code is volume-mounted
# or injected by the evaluation script).
ENV PYTHONPATH=/app

RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
    && git add README.md \
    && git commit -m "Initial commit"

##############################################
# EVALUATION ASSETS DIRECTORY
##############################################
# Populated at RUNTIME by the evaluation script.
RUN mkdir -p /eval_assets

CMD ["/bin/bash"]
```

Section 3: Working with run.sh and parsing.py

- **Do NOT** modify run.sh or parsing.py, specially the Do NOT modify sections. Only update the configure and parsing logic sections if required.
- These scripts are used for **baseline** (before) and **verification** (after) execution.
- Your job:
    - Baseline: Run on **empty** codebase → all unit tests must **FAIL** (not ERROR).
    - After Golden Patch: Run again → all same tests must **PASS**.

Typical usage (in Terminal, inside your project folder):

Bash

```bash
# Make scripts executable (only needed once)
chmod +x run.sh

# Run baseline or verification
./run.sh
python3 parsing.py   # or python parsing.py depending on setup
```

The output is usually a JSON file showing test results.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

```
root@2139670e9f6e:/app# bash run.sh
Running all tests...
============================= test session starts ==============================
platform linux -- Python 3.10.12, pytest-9.0.2, pluggy-1.6.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
rootdir: /app
configfile: pyproject.toml
plugins: cov-7.0.0
collecting ... collected 106 items

tests/test_clustering.py::TestClusteringContract::test_cluster_embeddings_returns_array_like PASSE
D [  0%]
tests/test_clustering.py::TestClusteringContract::test_cluster_labels_integer_like PASSED [  1%]
tests/test_clustering.py::TestClusteringContract::test_clustering_synthetic_dataset_expected_ident
ities PASSED [  2%]
tests/test_clustering.py::TestClusteringIdenticalEmbeddings::test_identical_embeddings_same_cluste
r PASSED [  3%]
tests/test_clustering.py::TestClusteringIncremental::test_incremental_clustering_optional_interfac
e PASSED [  4%]
tests/test_clustering.py::TestClusteringEdgeCases::test_empty_embeddings_raises PASSED [  5%]
tests/test_clustering.py::TestClusteringEdgeCases::test_none_raises PASSED [  6%]
tests/test_clustering.py::TestClusteringEdgeCases::test_wrong_shape_1d_raises PASSED [  7%]
tests/test_clustering.py::TestClusteringEdgeCases::test_large_batch_handled PASSED [  8%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_build_photo_groups_empty_i
nput_zero_photos PASSED [  9%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_organizer_output_allows_du
plicate_metadata PASSED [ 10%]
tests/test_edge_cases.py::TestEdgeCaseZeroFaces::test_zero_faces_returns_empty_list PASSED [ 11%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_extremely_small_image PASSED [ 12%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_low_res_face_crop_embedding PASSED [ 13
%]
tests/test_edge_cases.py::TestEdgeCaseMultipleFacesPerImage::test_multiple_faces_structure PASSED
[ 14%]
tests/test_edge_cases.py::TestEdgeCaseIdenticalEmbeddings::test_identical_embeddings_same_label PA
SSED [ 15%]
tests/test_edge_cases.py::TestEdgeCaseLargeBatch::test_large_batch_clustering PASSED [ 16%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_detection_none PASSED [ 16%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_embedding_none PASSED [ 17%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_clustering_empty PASSED [ 18%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_clustering_none PASSED [ 19%]
tests/test_embedding.py::TestEmbeddingContract::test_embedding_returns_ndarray PASSED [ 20%]
tests/test_embedding.py::TestEmbeddingContract::test_embedding_dimension_consistency PASSED [ 21%]
tests/test_embedding.py::TestEmbeddingContract::test_embedding_normalization_correctness PASSED [
22%]
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

codebase > {} test_results.json > [ ] tests > { } 5


```
{
  "tests": [
    {
      "name": "tests/test_clustering.py::TestClusteringContract::test_cluster_embeddings_return
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringContract::test_cluster_labels_integer_li
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringContract::test_clustering_synthetic_data
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringIdenticalEmbeddings::test_identical_embe
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringIncremental::test_incremental_clustering
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringEdgeCases::test_empty_embeddings_raises"
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringEdgeCases::test_none_raises",
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringEdgeCases::test_wrong_shape_1d_raises",
      "status": "PASSED"
```


CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

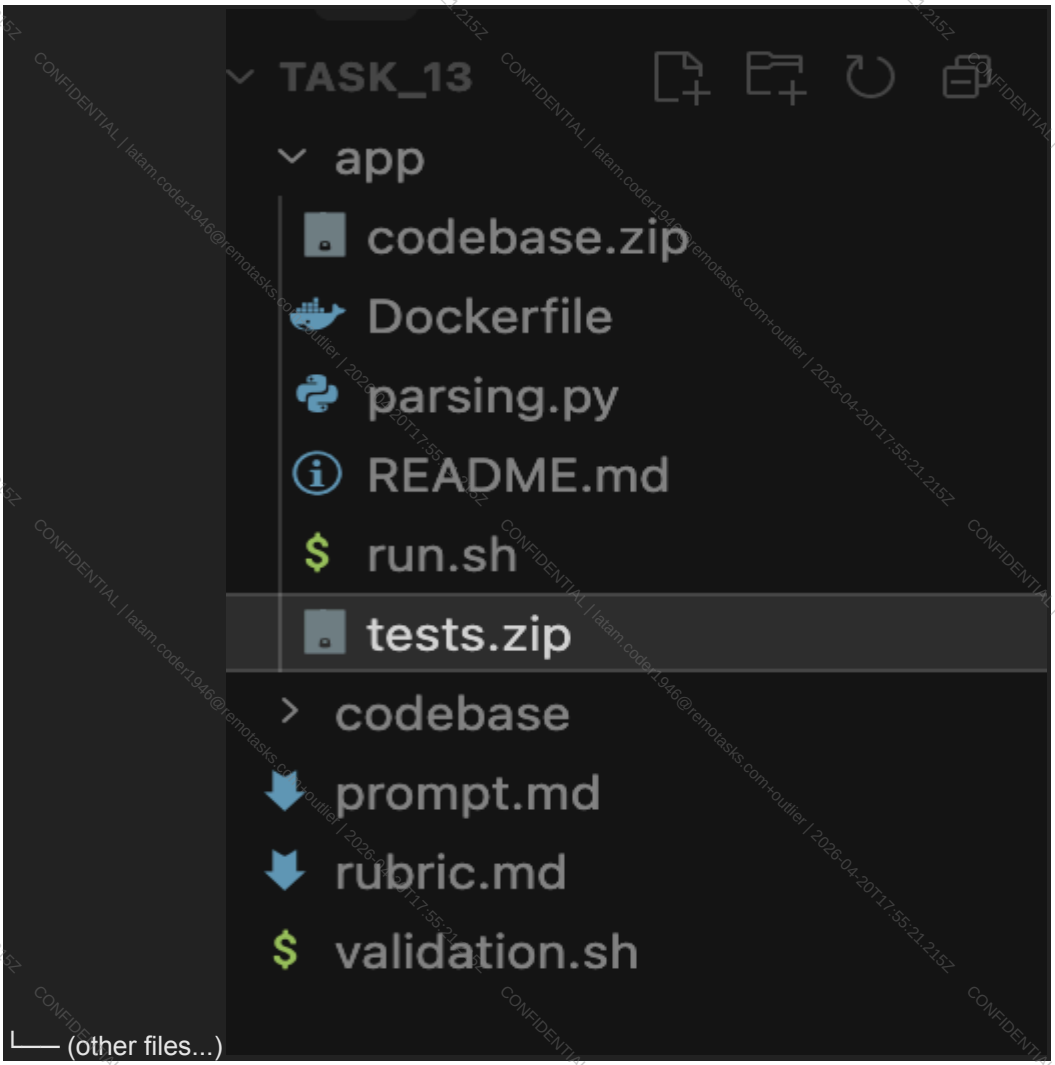
Section 4: Step-by-Step – Running Validation on Mac (using validation.sh)

Make sure **Docker Desktop** is running (whale icon visible in the menu bar).

Prepare your project folder structure

In Terminal, go to your main project directory (where you want to run validation):
```
cd ~/path/to/your/real-coder-project
```
Create a subfolder named exactly **app**:
```
mkdir app
```
Your folder should now look like this:
```
your-real-coder-project/

├── app/                ← put the 5 required files here

│   ├── Dockerfile

│   ├── tests.zip

│   ├── codebase.zip

│   ├── run.sh

│   └── parsing.py

├── validation.sh       ← the evaluation script (outside app/)
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z



Place the required files

1. Copy/move the following files **into** the app/ folder:
    - Dockerfile (your modified version with only dependency additions)
    - tests.zip (the hidden test suite provided by the task)
    - codebase.zip (your Golden Patch / agent's submitted code as a zip)
    - run.sh (the test runner script)
    - parsing.py (the result parser script)

Make sure **no extra files** are in app/ unless explicitly allowed.

Place and prepare validation.sh

Save the provided validation.sh script **outside** the app/ folder (in the project root).

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Make it executable:

```
chmod +x validation.sh
```

Open validation.sh in a text editor (e.g. nano, VS Code, TextEdit) and **update the APP_DIR line** to point to your actual app folder path. Example:

```
# Change this line to match your real path

APP_DIR="/Users/yourusername/path/to/your-real-coder-project/app"
```

Or, if you're running from the parent folder:

Bash

```
APP_DIR="$(pwd)/app"
```

Save the file.

Run the validation

2. From the project root (where validation.sh is located), execute:

```
bash validation.sh
```

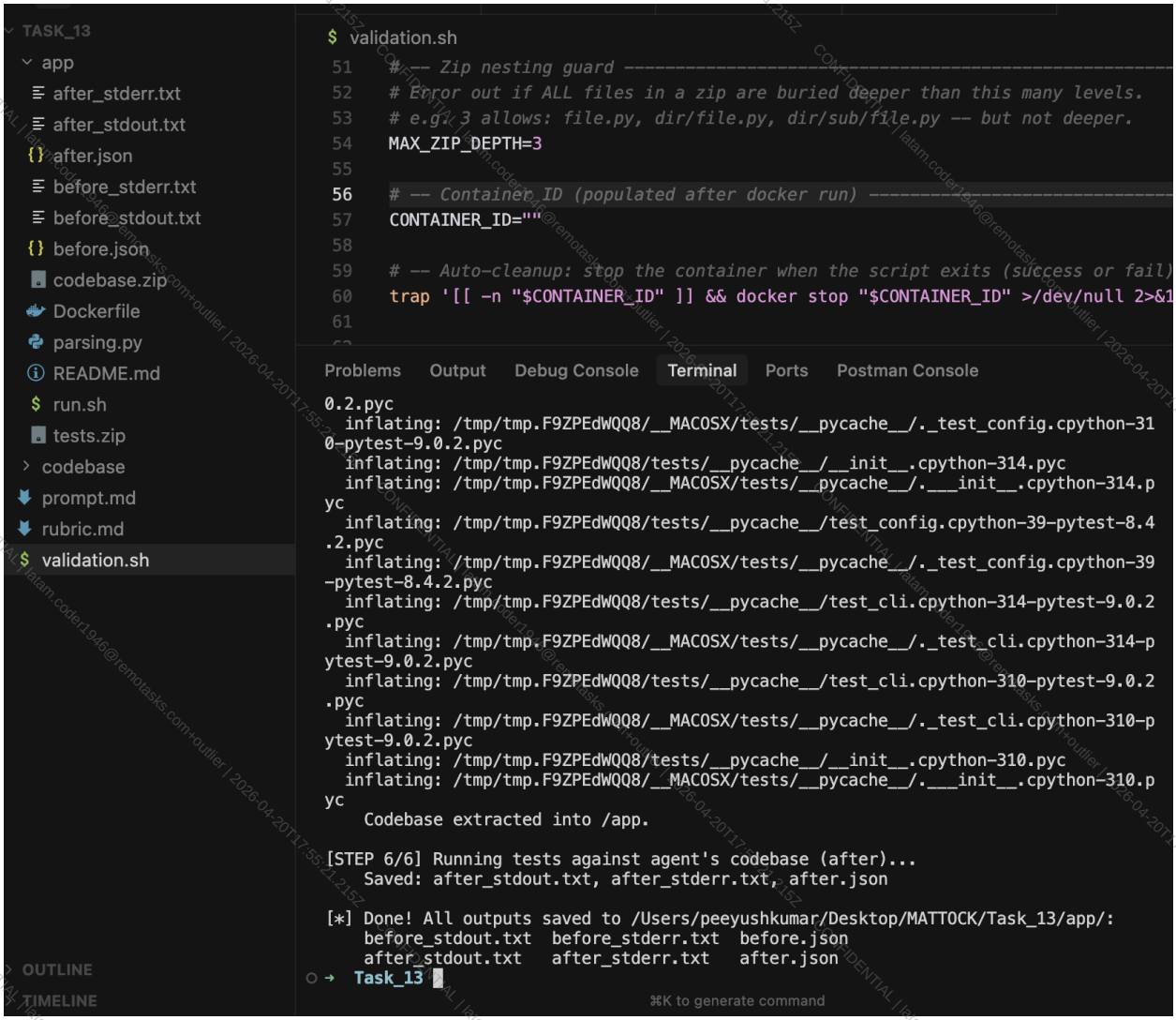
   - The script will:
     - Build the Docker image from app/Dockerfile
     - Start a container
     - Inject tests & scripts
     - Run tests **before** (empty /app) → produces before_*.txt and before.json
     - Inject your codebase.zip into /app
     - Run tests **after** → produces after_*.txt and after.json
     - Clean up automatically

All 6 output files will appear **inside** the app/ folder:

text

```
app/

├── before_stdout.txt

├── before_stderr.txt

├── before.json

├── after_stdout.txt

├── after_stderr.txt

└── after.json
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z



Check results

3. Open the JSON files:
   - before.json → should show most/all tests **FAILED** (baseline with empty code)
   - after.json → should show tests **PASSED** if your Golden Patch is correct
4. Goal reminder:
   - Before fix → JSON shows **all (or nearly all) tests FAILED**
   - After fix → JSON shows **all relevant tests PASSED**
5. If validation fails (e.g. files missing, zip nesting too deep, Docker build error), read the error messages carefully and fix accordingly.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 5: Step-by-Step – Running run.sh and parsing.py in codebase on Mac

1. Make sure **Docker Desktop** is running (whale icon in menu bar).
2. Navigate to your project folder in Terminal:
   `cd ~/path/to/your/real-coder-project`
3. **Build** the Docker image (do this after any Dockerfile change):
   `docker build -t real-coder-env .`
   - -t real-coder-env = friendly name for your image.
   - . = current directory (must contain Dockerfile).
4. **Run** the container interactively in codebase:
   `docker run -it --rm -v $(pwd):/app real-coder-env bash`
   - -v $(pwd):/app mounts your current folder into /app inside container.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

- You can now run ./run.sh or python3 parsing.py inside.

```
● → Task_13 cd codebase
● → codebase docker build -t real-coder-env .
[+] Building 2.0s (11/11) FINISHED
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 74B                                           0.0s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [internal] load metadata for docker.io/library/ubuntu:24.04               1.9s
 => [1/7] FROM docker.io/library/ubuntu:24.04@sha256:d1e2e92c075e5ca139d51a14  0.0s
 => CACHED [2/7] RUN apt-get update && apt-get install -y     git     curl    0.0s
 => CACHED [3/7] RUN python3.14 -m venv /opt/venv     && /opt/venv/bin/pip in  0.0s
 => CACHED [4/7] RUN ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python3   0.0s
 => CACHED [5/7] WORKDIR /app                                                 0.0s
 => CACHED [6/7] RUN git init     && git config --global user.email "agent@ex  0.0s
 => CACHED [7/7] RUN mkdir -p /eval_assets                                    0.0s
 => exporting to image                                                        0.0s
 => => exporting layers                                                       0.0s
 => => writing image sha256:7e1cd382069aa4553e40e018166edab542275120212d6086f  0.0s
 => => naming to docker.io/library/real-coder-env                             0.0s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn
 how to fix them
○ → codebase docker run -it --rm -v $(pwd):/app real-coder-env bash
root@d957dcd1a8d8:/app# bash run.sh
Running all tests...
============================= test session starts ==============================
platform linux -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- /opt/venv/bin/python3
.14
cachedir: .pytest_cache
rootdir: /app
configfile: pyproject.toml
collecting ... collected 78 items

tests/test_cli.py::TestCLIEntryPoint::test_main_is_click_command PASSED  [  1%]
tests/test_cli.py::TestCLIEntryPoint::test_cli_invocation_with_config_path PASSED [
 2%]
tests/test_cli.py::TestCLIEntryPoint::test_cli_accepts_filter_keyword_flag PASSED [
 3%]
tests/test_cli.py::TestCLIEntryPoint::test_cli_accepts_filter_level_flag PASSED [  5
```

```
tests/test_ssh_client.py::TestSSHClientManager::test_SSHClientManager_tails_specifie
d_log_file PASSED [ 76%]
tests/test_ssh_client.py::TestSSHClientManager::test_SSHClientManager_accepts_option
al_label PASSED [ 78%]
tests/test_ssh_client.py::TestSSHClientManager::test_SSHClientManager_has_connect_me
thod PASSED [ 79%]
tests/test_ssh_client.py::TestSSHClientManager::test_SSHClientManager_has_close_meth
od PASSED [ 80%]
tests/test_ssh_client.py::TestSSHClientManager::test_SSHClientManager_has_stream_met
hod PASSED [ 82%]
tests/test_ssh_client.py::TestSSHClientManager::test_stream_executes_tail_F_command
PASSED [ 83%]
tests/test_ssh_client.py::TestSSHClientManager::test_stream_uses_label_as_identifier
_when_set PASSED [ 84%]
tests/test_ssh_client.py::TestSSHClientManager::test_stream_falls_back_to_name_when_
label_is_none PASSED [ 85%]
tests/test_ssh_client.py::TestSSHClientManager::test_close_shuts_down_ssh_client PAS
SED [ 87%]
tests/test_ssh_client.py::TestSSHClientManager::test_close_is_idempotent PASSED [ 88
%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_accepts_connections_list PA
SSED [ 89%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_accepts_filters PASSED [ 91
%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_multiple_servers PASSED [ 9
2%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_returns_none PASSED [ 93%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_spawns_one_thread_per_conne
ction PASSED [ 94%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_calls_apply_filter_per_line
 PASSED [ 96%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_calls_format_line_for_match
ing_lines PASSED [ 97%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_calls_close_on_all_connecti
ons PASSED [ 98%]
tests/test_streamer.py::TestStreamLogs::test_stream_logs_uses_lock_for_stdout PASSED
 [100%]

============================== 78 passed in 10.22s ==============================

Results written to: /app/results.json
```

5. For validation (most common):
    - Apply your code changes / Golden Patch.
    - Re-build image if Dockerfile changed: docker build -t real-coder-env .
    - Run validation scripts as instructed (usually via ./run.sh and parsing.py).
6. Goal reminder:
    - Before fix → JSON shows **all tests FAILED**.
    - After fix → JSON shows **all tests PASSED**.

Section 6: Troubleshooting Tips on Mac

- Docker not starting? → Restart Docker Desktop from menu bar → Quit & Relaunch.
- Build fails with permission error? → Make sure you accepted Docker permissions.
- Slow build on M1/M2/M3 Mac? → Normal for first build (caching helps later).

CONFIDENTIAL | latam.coder.ai@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

- "docker: command not found"? → Open Docker Desktop first, or restart Terminal.
- Need to stop everything? → Quit Docker Desktop from menu bar.

End of Guide

Follow these rules exactly to avoid breaking validation. Good luck with your Real Coder submissions! 🐸

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

For Windows

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

For Windows

Real Coder – Docker & Validation Instructions for Windows Users

Document Purpose

This guide explains how to safely modify the **Dockerfile**, work with **run.sh** and **parsing.py**, and run the validation process on a **Windows** computer.

This project has some strict rules as mentioned below:

- only allowed modifications in specific places,
- no Dockerfile `COPY`,
- no runtime dependency installs in `run.sh`,
- and correct before/after JSON generation for validation.

Important Warnings

Follow these carefully to avoid breaking validation:

- **Do not modify `validation.sh` except (refer Attempter's instruction):**
  - The main root file path if needed, defaults to /app
  - Can add these two lines if script doesn't run: - /bin/bash -c 'parse_results stdout.txt stderr.txt before.json' || true - /bin/bash -c 'parse_results stdout.txt stderr.txt after.json' || true
- **Do not modify protected blocks** in `run.sh` / `parsing.py` (example: `### COMMON EXECUTION; DO NOT MODIFY ###`).
- **Do not install dependencies at runtime** in `run.sh`.
- **Do not use `COPY` in Dockerfile** (image should be built for environment/dependencies only; code is mounted with volumes).

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 1: Prerequisites on Windows

1) Install Docker Desktop (required)

- Download Docker Desktop for Windows.
- During install, keep **WSL2 backend** enabled (recommended).
- Start Docker Desktop and wait until it shows Docker is running.

2) Install a terminal you will use

Use **one** of these (recommended order):

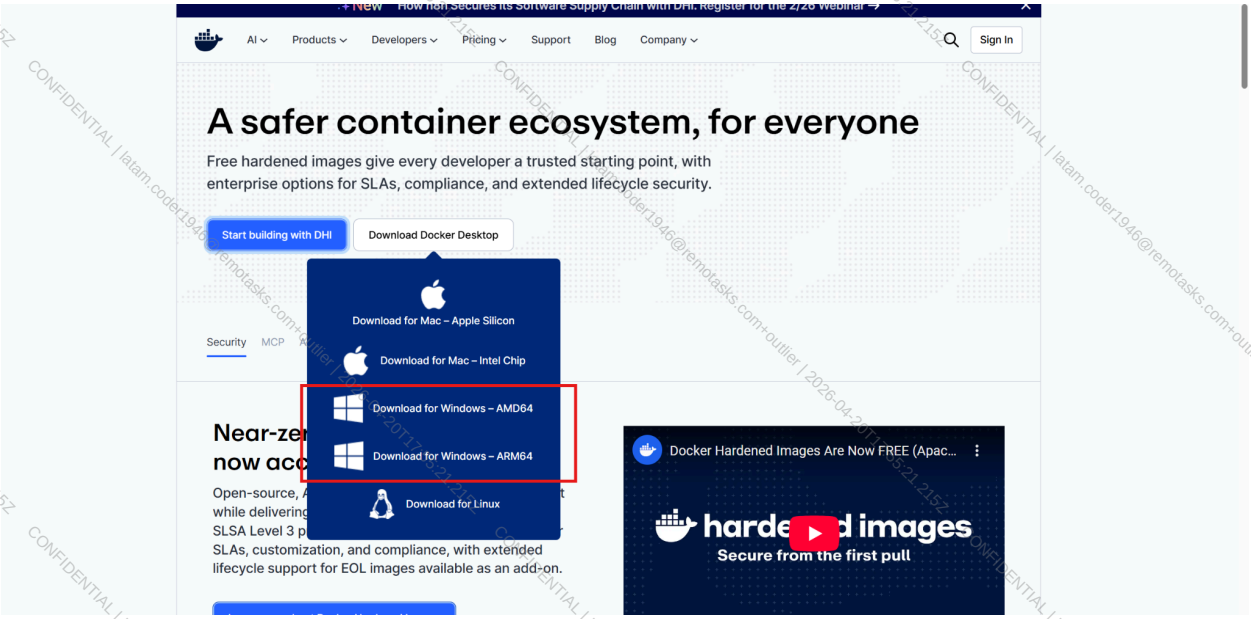
1. **PowerShell** (works fine)
2. **Git Bash** (often easiest for shell-style commands)
3. **WSL Ubuntu shell** (also good, if already installed)

3) Verify Docker works

Open PowerShell (or Git Bash) and run:

docker --version

If Docker is not running, open Docker Desktop first.



```
PS D:\> docker --version
Docker version 27.2.0, build 3ab4256
PS D:\>
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 2: Allowed & Forbidden Modifications to Dockerfile

Use only the provided Dockerfile template.

Allowed changes

- Add system packages in the **SYSTEM DEPENDENCIES** section (via `apt-get install`).
- Add Python dependencies with `pip install`.
- Add framework/runtime dependencies needed by the app/tests (Python, Node, DB clients, etc.).
- You may hardcode dependencies (example: `pip install pyyaml requests pytest`), especially when tests import them.

Required rule

- **Install all dependencies in Dockerfile** (build-time), not in `run.sh`.

Forbidden changes (Do NOT do these)

- Do not change Dockerfile structure/order/comments unless explicitly allowed.
- Do not remove existing lines.
- Do not add `COPY` or `ADD` for project code.
- Do not change base image / `WORKDIR /app` / default setup behavior.
- Do not change entrypoint/default command unexpectedly.

Dockerfile reminder for this workflow

Your Dockerfile is for:

- environment setup,
- package installs,
- shell/tools readiness.

Your code/tests are run using **volume mounts**, not baked into the image.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

```
##############################################
# BASE IMAGE
##############################################
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

##############################################
# SYSTEM DEPENDENCIES
##############################################
RUN apt-get update && apt-get install -y \
    git \
    python3 \
    python3-pip \
    python3-setuptools \
    python-is-python3 \
    unzip \
    && rm -rf /var/lib/apt/lists/*

##############################################
# WORKING DIRECTORY + GIT SETUP
##############################################
WORKDIR /app

RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
    && git add README.md \
    && git commit -m "Initial commit"

##############################################
# EVALUATION ASSETS DIRECTORY
##############################################
# Populated at RUNTIME by the evaluation script.
RUN mkdir -p /eval_assets

CMD ["/bin/bash"]
```

src: template docker file (from task taxonomy)

```
app > 🐳 Dockerfile > ...
1   ##############################################
2   # BASE IMAGE
3   ##############################################
4   FROM ubuntu:24.04 (last pushed 1 week ago)
5
6   ENV DEBIAN_FRONTEND=noninteractive
7
8   ##############################################
9   # SYSTEM DEPENDENCIES (minimal to avoid disk-full on build)
10  # python3.14-venv only; no -dev to avoid pulling build-essential/gcc
11  ##############################################
12  RUN apt-get update && apt-get install -y \
13      git \
14      curl \
15      unzip \
16      openssh-client \
17      software-properties-common \
18      && add-apt-repository ppa:deadsnakes/ppa \
19      && apt-get update && apt-get install -y \
20      python3.14 \
21      python3.14-venv \
22      python3-pip \
23      && rm -rf /var/lib/apt/lists/*
24
25  ##############################################
26  # CREATE VENV AT /opt/venv WITH ALL DEPENDENCIES
27  ##############################################
28  RUN python3.14 -m venv /opt/venv \
29      && /opt/venv/bin/pip install --upgrade pip "setuptools>=61.0" \
30      && /opt/venv/bin/pip install --no-cache-dir \
31          "click==8.0" \
32          "paramiko>=3.0" \
33          "pyyaml>=6.0" \
34          "rich>=13.0" \
35          "pytest>=9.0.2"
36
37  ENV PATH="/opt/venv/bin:${PATH}"
38
39  # Point python3 and python to the venv interpreter so run.sh's
40  # fallback (python3) always resolves to the venv with all packages.
41  RUN ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python3 \
42      && ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python
43
44  ##############################################
45  # WORKING DIRECTORY + GIT SETUP
46  ##############################################
47  WORKDIR /app
48
49  # PYTHONPATH includes /app so the log_aggregator package is importable
50  # once code is injected into /app at runtime (no COPY — code is volume-mounted
51  # or injected by the evaluation script).
52  ENV PYTHONPATH=/app
53
54  RUN git init \
55      && git config --global user.email "agent@example.com" \
56      && git config --global user.name "Agent" \
57      && echo "# Workspace" > README.md \
58      && git add README.md \
59      && git commit -m "Initial commit"
```

```
app > 🐳 Dockerfile > ...
18      && add-apt-repository ppa:deadsnakes/ppa \
19      && apt-get update && apt-get install -y \
20      python3.14 \
21      python3.14-venv \
22      python3-pip \
23      && rm -rf /var/lib/apt/lists/*
24
25  ##############################################
26  # CREATE VENV AT /opt/venv WITH ALL DEPENDENCIES
27  ##############################################
28  RUN python3.14 -m venv /opt/venv \
29      && /opt/venv/bin/pip install --upgrade pip "setuptools>=61.0" \
30      && /opt/venv/bin/pip install --no-cache-dir \
31          "click==8.0" \
32          "paramiko>=3.0" \
33          "pyyaml>=6.0" \
34          "rich>=13.0" \
35          "pytest>=9.0.2"
36
37  ENV PATH="/opt/venv/bin:${PATH}"
38
39  # Point python3 and python to the venv interpreter so run.sh's
40  # fallback (python3) always resolves to the venv with all packages.
41  RUN ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python3 \
42      && ln -sf /opt/venv/bin/python3.14 /usr/local/bin/python
43
44  ##############################################
45  # WORKING DIRECTORY + GIT SETUP
46  ##############################################
47  WORKDIR /app
48
49  # PYTHONPATH includes /app so the log_aggregator package is importable
50  # once code is injected into /app at runtime (no COPY — code is volume-mounted
51  # or injected by the evaluation script).
52  ENV PYTHONPATH=/app
53
54  RUN git init \
55      && git config --global user.email "agent@example.com" \
56      && git config --global user.name "Agent" \
57      && echo "# Workspace" > README.md \
58      && git add README.md \
59      && git commit -m "Initial commit"
60
61  ##############################################
62  # EVALUATION ASSETS DIRECTORY
63  ##############################################
64  # Populated at RUNTIME by the evaluation script.
65  RUN mkdir -p /eval_assets
66
67  CMD ["/bin/bash"]
68
```

src: modified docker file

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 3: Working with `run.sh` and `parsing.py` (Windows)

These scripts are used for both:

- **Before (baseline)** → tests should fail (FAILED, not generic ERROR if per-test results exist)
- **After (verification)** → the same tests should pass

Do not break the test output contract

`parsing.py` must produce JSON like:

```
{
  "tests": [
    { "name": "tests/test_auth.py::test_login", "status": "PASSED" },
    { "name": "tests/test_auth.py::test_invalid_email", "status": "FAILED" },
    { "name": "tests/test_auth.py::test_rate_limit", "status": "FAILED" }
  ]
}
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Windows-specific caution

On Windows:

- `run.sh` is a **Linux shell script** and must be executed **inside a Docker container** (Linux container).
- So keep it Unix-compatible and LF-only.
- Do not try to convert `run.sh` to `.bat` or PowerShell.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 4: Step-by-Step – Running Validation and Run.sh on windows

Part A: Build the Docker image (Setup 1 / codebase flow)

Go to your **project root** (the folder that contains `codebase/`).

PowerShell

```
docker build -t real-coder-env .\codebase
```

Git Bash

```
docker build -t real-coder-env ./codebase
```

This builds the image from the `codebase/Dockerfile`.

Rebuild the image whenever you change the Dockerfile.

Part B: Run `run.sh` in Docker

This is the command you'll use most often. It should:

- mount your codebase into `/app`,
- run `./run.sh`,

PowerShell (recommended)

```
# from project root

Task_{task_number}  # project root

   /codebase/
    |-- Dockerfile          # Docker image definition
    |-- /tests/             # test suite
    |-- … project files     # Agent's submitted code files and dir
    |-- run.sh              # Test runner script
    `-- parsing.py          # Parses test output into JSON
  /validation.sh            # Validation script to run
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

```
docker rm -f rc-test-run 2>$null | Out-Null

docker run --name rc-test-run  -v "${PWD}\codebase:/app"   -w /app  -it real-coder-env  bash -lc "chmod +x run.sh"

docker rm -f rc-test-run
```

This setup shows test output directly in the terminal and still saves the final JSON.

```
============================= test session starts ==============================
platform linux -- Python 3.10.12, pytest-9.0.2, pluggy-1.6.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
rootdir: /app
configfile: pyproject.toml
plugins: cov-7.0.0
collecting ... collected 106 items

tests/test_clustering.py::TestClusteringContract::test_cluster_embeddings_returns_array_like PASSE
D [  0%]
tests/test_clustering.py::TestClusteringContract::test_cluster_labels_integer_like PASSED [  1%]
tests/test_clustering.py::TestClusteringContract::test_clustering_synthetic_dataset_expected_ident
ities PASSED [  2%]
tests/test_clustering.py::TestClusteringIdenticalEmbeddings::test_identical_embeddings_same_cluste
r PASSED [  3%]
tests/test_clustering.py::TestClusteringIncremental::test_incremental_clustering_optional_interfac
e PASSED [  4%]
tests/test_clustering.py::TestClusteringEdgeCases::test_empty_embeddings_raises PASSED [  5%]
tests/test_clustering.py::TestClusteringEdgeCases::test_none_raises PASSED [  6%]
tests/test_clustering.py::TestClusteringEdgeCases::test_wrong_shape_1d_raises PASSED [  7%]
tests/test_clustering.py::TestClusteringEdgeCases::test_large_batch_handled PASSED [  8%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_build_photo_groups_empty_i
nput_zero_photos PASSED [  9%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_organizer_output_allows_du
plicate_metadata PASSED [ 10%]
tests/test_edge_cases.py::TestEdgeCaseZeroFaces::test_zero_faces_returns_empty_list PASSED [ 11%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_extremely_small_image PASSED [ 12%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_low_res_face_crop_embedding PASSED [ 13
%]
tests/test_edge_cases.py::TestEdgeCaseMultipleFacesPerImage::test_multiple_faces_structure PASSED
[ 14%]
tests/test_edge_cases.py::TestEdgeCaseIdenticalEmbeddings::test_identical_embeddings_same_label PA
SSED [ 15%]
tests/test_edge_cases.py::TestEdgeCaseLargeBatch::test_large_batch_clustering PASSED [ 16%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_detection_none PASSED [ 16%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_embedding_none PASSED [ 17%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_clustering_empty PASSED [ 18%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_clustering_none PASSED [ 19%]
tests/test_embedding.py::TestEmbeddingContract::test_embedding_returns_ndarray PASSED [ 20%]
tests/test_embedding.py::TestEmbeddingContract::test_embedding_dimension_consistency PASSED [ 21%]
tests/test_embedding.py::TestEmbeddingContract::test_embedding_normalization_correctness PASSED [
22%]
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

codebase > {} test_results.json > [ ] tests > { } 5


```
{
  "tests": [
    {
      "name": "tests/test_clustering.py::TestClusteringContract::test_cluster_embeddings_return
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringContract::test_cluster_labels_integer_li
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringContract::test_clustering_synthetic_data
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringIdenticalEmbeddings::test_identical_embe
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringIncremental::test_incremental_clustering
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringEdgeCases::test_empty_embeddings_raises"
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringEdgeCases::test_none_raises",
      "status": "PASSED"
    },
    {
      "name": "tests/test_clustering.py::TestClusteringEdgeCases::test_wrong_shape_1d_raises",
      "status": "PASSED"
```

Part C: Run validation flow

```
/app/
 |-- Dockerfile          # Docker image definition
 |-- tests.zip           # Hidden test suite
 |-- codebase.zip        # Agent's submitted code
 |-- run.sh              # Test runner script
 `-- parsing.py          # Parses test output into JSON
/validation.sh           # Validation script to run
```

This is the evaluator-style structure (where `validation.sh` is used).

Key rule

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

- **Do not modify `validation.sh` except (refer Attempter's instruction):**
  - The main root file path if needed, defaults to /app (APP_DIR)
  - Can add these two lines if script doesn't run: - /bin/bash -c 'parse_results stdout.txt stderr.txt before.json' || true - /bin/bash -c 'parse_results stdout.txt stderr.txt after.json' || true
- Make sure your `run.sh`, `parsing.py`, and Dockerfile are same as your project's root file which were used earlier

1. **Typical Windows approach (PowerShell)**

Run from the folder that contains the evaluator files (Dockerfile, validation.sh, codebase.zip/tests.zip or mounted equivalents):

```
docker build -t real-coder-env .
```

```
docker run --rm -it -v "${PWD}:/app" -w /app real-coder-env bash -lc "./validation.sh"
```

If your validation script expects a sibling path or different mount layout, use the exact folder layout required by your task, but keep the same pattern:

  - mount correct host folder(s),
  - run `./validation.sh`,
  - let `run.sh` write `before.json` / `after.json`.

2. WSL approach:
   - **Install Ubuntu on WSL (Windows)**

     1) Open PowerShell as Administrator and run:

     ```
     wsl --install -d Ubuntu
     ```

     2) Reboot (if prompted)

     If Windows asks you to reboot, restart your system.

     3) Complete Ubuntu first-time setup

     After reboot:

     - Open **Ubuntu** from the Start menu
     - Complete the first-time setup (create username/password)

     4) Verify WSL + Ubuntu

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Run these commands in PowerShell:

i. wsl --list --verbose
ii. wsl --update
iii. wsl --set-default-version 2

5) Install Docker Desktop and Enable WSL Integration

1) Install Docker Desktop for Windows

2) In Docker Desktop, enable these settings:

- **Settings** → **General** → Enable **Use WSL 2 based engine**
- **Settings** → **Resources** → **WSL Integration** → Enable integration for your **Ubuntu** distro

Open Ubuntu (WSL) and Verify Docker Works

In the Ubuntu terminal, run:

- docker version

If this fails:

- Make sure **Docker Desktop is running**
- Make sure **WSL Integration is enabled for Ubuntu** in Docker Desktop settings

6) Run `validate.sh` / `validation.sh` from WSL (Full Flow)

Run this from the folder that contains the validation script:

- bash ./validation.sh

Or, if your file is named `validate.sh`:

- bash ./validate.sh

Important

Run it from the **same directory** where the script expects its files/folders (for example, where `validation.sh` and the sibling `/app` folder live)

Do not modify `validation.sh`

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 5: Running the Whole Application (Windows)

Use this only when you want to launch the app (not just run tests).

Because every project differs (Python app / Node app / frontend-backend monorepo), use your project's actual start command inside Docker.

Example pattern (PowerShell)

```
docker run --rm -it `
  -v "${PWD}\codebase:/app" `
  -w /app `
  -p 3000:3000 -p 8000:8000 `
  real-coder-env `
  bash
```

Then inside the container, run the real app command, for example:

- `python3 -m <your_app>`
- `uvicorn app.main:app --host 0.0.0.0 --port 8000`
- `npm run dev`
- `npm run start`
- frontend + backend startup commands (if monorepo)

Replace placeholders with your actual project commands.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

Section 6: Troubleshooting Tips on Windows

1) `python3\r: No such file or directory`

Cause: CRLF line endings in `run.sh` or `parsing.py`.

Fix:

- convert file to LF,
- keep `.gitattributes` in repo root.

---

2) Docker mount path errors in PowerShell

Use:

-v "${PWD}\codebase:/app"

(not Unix-style `$PWD/codebase` in plain PowerShell)

If using Git Bash, use:

-v "$PWD/codebase:/app"

---

3) Tests not found / code not found in validation

This is usually a path-detection issue in `run.sh`.

Make sure `run.sh`:

- detects `/eval_assets/tests` and `/app/tests`,
- uses `SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"`,
- writes JSON to the correct file path,
- and invokes `parsing.py` itself.

---

4) Build passes but runtime import errors happen

Cause: dependency missing in Dockerfile.

Fix:

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z

- scan tests for imports (`yaml`, `requests`, `click`, etc.),
- pre-install them in Dockerfile,
- rebuild image.

---

5) Validation build errors with vague message

Most common cause: Dockerfile contains a `COPY` command.

Fix:

- remove `COPY`,
- rebuild image,
- mount code with `-v` instead.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.215Z